Programación Orientada a Objetos
Herencia y Polimorfismo

Dificultad: *Básica – Media*
Tiempo aproximado: *60 minutos*

Hecho por: Matías González
Total: Hasta 1 punto

Índice

Índice **2**
Kat-Kingdom **3**
– Contexto – 3
– Información necesaria – 4
– Métodos especializados – 5
– Validaciones esperadas – 6
– Operaciones del sistema – 7
– Requerimientos de la clase principal – 8
– Resultado esperado – 9
– Diagrama de Clases – 10
Rúbrica de Evaluación – Kat Kingdom 11
¡Ánimo Chicos! **12**

Kat-Kingdom

– Contexto –

En el misterioso mundo de **Kat Kingdom**, un reino gobernado por gatos mágicos, la paz depende de valientes aventureros capaces de enfrentarse a criaturas oscuras, explorar territorios desconocidos y proteger a sus habitantes de las amenazas que acechan más allá de sus fronteras.

Durante siglos, los gatos del reino han desarrollado habilidades extraordinarias gracias a su conexión con la magia. Sin embargo, no todos poseen las mismas capacidades de combate. Algunos prefieren utilizar poderosas armas encantadas, mientras que otros dedican su vida al estudio de antiguos hechizos.

Para preparar a las nuevas generaciones de aventureros, el reino fundó la **Academia de Gatos Mágicos**, una institución encargada de entrenar y registrar a los gatos que desean convertirse en héroes. Actualmente, la academia reconoce dos especializaciones principales:

- **Gatos Guerreros:** Son expertos en el combate cuerpo a cuerpo. Utilizan espadas encantadas que les permiten aumentar su fuerza durante las batallas. El poder de cada guerrero depende tanto de sus habilidades naturales como del poder adicional que proporciona su arma.
- **Gatos Hechiceros:** Son especialistas en el uso de la magia. A través de sus conocimientos, pueden activar poderosos hechizos que duplican temporalmente su poder de combate. Sin embargo, cuando sus hechizos se encuentran inactivos, solamente pueden utilizar su poder natural.

A pesar de sus diferencias, todos los gatos comparten características fundamentales, como su **nombre y poder base**. No obstante, el cálculo de su poder final depende de la especialización a la que pertenecen.

Con el creciente número de aventureros, la academia ha comenzado a experimentar dificultades para mantener un registro organizado de sus integrantes y determinar correctamente sus capacidades de combate.

Por este motivo, los administradores de **Kat Kingdom** han solicitado desarrollar un sistema en Java que permita **registrar gatos mágicos, almacenar su información y calcular sus poderes finales**, considerando las habilidades particulares de cada especialización.

El sistema deberá ser capaz de administrar distintos tipos de gatos sin necesidad de tratarlos por separado, permitiendo que cada personaje determine su propio poder de combate según las reglas establecidas por la academia.

Para resolver esta situación, deberás aplicar los principios de la **Programación Orientada a Objetos**, utilizando clases abstractas, herencia, encapsulamiento, sobreescritura de métodos, polimorfismo y colecciones mediante `ArrayList`.

– Información necesaria –

El sistema contempla las siguientes clases:

– GatoMágico

Todos los gatos del reino comparten los siguientes atributos privados:

- `nombre` (String)
- `poderBase` (int)

La clase debe incluir:

- Constructor que reciba ambos atributos.
- Métodos `getNombre()` y `getPoderBase()`.
- Método `setPoderBase()` para asignar y validar el poder base.
- Método abstracto `calcularPoderFinal()`, que retorne un valor entero (`int`).

El constructor debe utilizar `setPoderBase()` para asignar el poder recibido, asegurando que la validación se aplique desde la creación del objeto.

– GatoGuerrero

Esta clase debe heredar de `GatoMagico` y agregar el siguiente atributo privado:

- `bonusEspada` (int)

Debe incluir:

- Constructor que reciba nombre, poder base y bonus de espada.
- Uso de `super()` para inicializar los atributos heredados.
- Sobreescritura del método `calcularPoderFinal()`.

– GatoHechicero

Esta clase debe heredar de `GatoMagico` y agregar el siguiente atributo privado:

- `hechizoActivo` (boolean)

Debe incluir:

- Constructor que reciba nombre, poder base y estado del hechizo.
- Uso de `super()` para inicializar los atributos heredados.
- Sobreescritura del método `calcularPoderFinal()`.

– Métodos especializados –

El sistema debe permitir calcular el poder final de cualquier gato registrado, independientemente de su tipo.

Cada clase determinará este valor según sus propias reglas.

1. GatoGuerrero

El poder final corresponde a la suma del poder base y el bonus de espada.

Poder final = Poder base + Bonus de espada

Ejemplo:

- Poder base: 50
- Bonus de espada: 15
- **Poder final: 65**

2. GatoHechicero

El poder final dependerá de si el hechizo se encuentra activo.

- Si `hechizoActivo` es `true`, el poder base se multiplica por 2.
- Si `hechizoActivo` es `false`, el poder final será igual al poder base.

Ejemplo:

- Poder base: 40
- Hechizo activo: true
- **Poder final: 80**

Ambas clases deben implementar estas reglas sobrescribiendo el método `calcularPoderFinal()` mediante `@Override`.

– Validaciones esperadas –

La clase `GatoMagico` debe implementar una validación en el método `setPoderBase()`.

Regla:

- **El atributo `poderBase` debe ser mayor que cero.**

Si se intenta asignar un valor menor o igual a cero, el método deberá lanzar una excepción de tipo **`IllegalArgumentException`**, incluyendo un mensaje descriptivo.

La validación debe ejecutarse también al crear un gato mediante su constructor.

> **Importante:** Esta será la única validación obligatoria del ejercicio. No se requiere implementar `try-catch`.

Clase RegistroGatos (Colección)

La academia necesita mantener un registro de todos los gatos que participan en sus actividades.

Para ello, deberás crear una clase llamada `RegistroGatos`, encargada de administrar una colección de objetos mediante:

```
ArrayList<GatoMagico>
```

Esta clase debe contar con:

Constructor

- Inicializar el `ArrayList` que almacenará los gatos.

Método agregarGato()

- Recibir un objeto de tipo `GatoMagico`.
- Agregarlo a la colección utilizando `add()`.

Método mostrarGatos()

- Recorrer todos los gatos registrados mediante un ciclo.
- Mostrar el nombre de cada gato.
- Mostrar su poder base.
- Mostrar su poder final utilizando `calcularPoderFinal()`.

Se debe aprovechar el **polimorfismo**, de manera que el sistema pueda trabajar tanto con `GatoGuerrero` como con `GatoHechicero` utilizando referencias de tipo `GatoMagico`.

No se permite utilizar `instanceof` para determinar qué cálculo realizar.

– Requerimientos de la clase principal –

Crear una clase `Main` que permita demostrar el funcionamiento del sistema.

La clase principal deberá:

1. Crear un objeto de tipo `RegistroGatos`.
2. Instanciar los tres gatos mágicos utilizando los datos proporcionados.
3. Registrar los gatos mediante `agregarGato()`.
4. Invocar `mostrarGatos()` para visualizar todos los personajes y sus poderes finales.

Datos para la clase principal

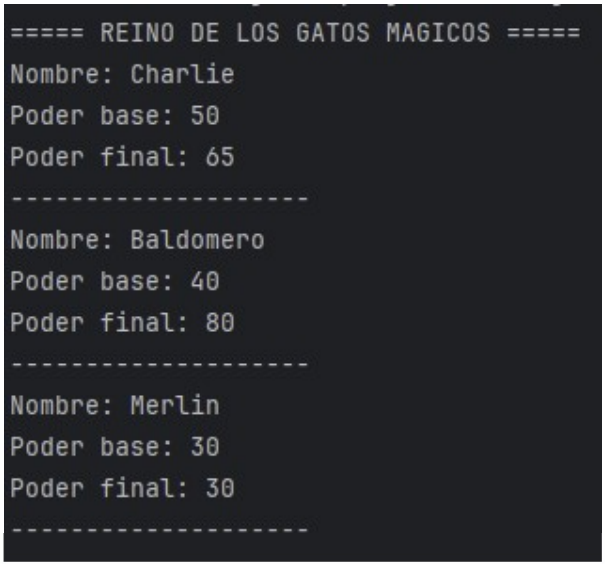
Tipo	Nombre	Poder base	Atributo específico
GatoGuerrero	**Charlith**	50	Bonus espada: 15
GatoHechicero	**Merlin**	40	Hechizo activo: true
GatoHechicero	**Baldomero**	30	Hechizo activo: false

Consideraciones:

- Cada clase debe estar en su propio archivo `.java`.
- Los atributos deben ser privados.
- Se debe utilizar correctamente `extends`, `super()` y `@Override`.
- No se requiere ingresar información mediante teclado.
- No se requiere implementar un menú interactivo.
- No se requieren interfaces ni operaciones adicionales sobre el `ArrayList`.
- El programa debe compilar y ejecutarse correctamente.

– Resultado esperado –

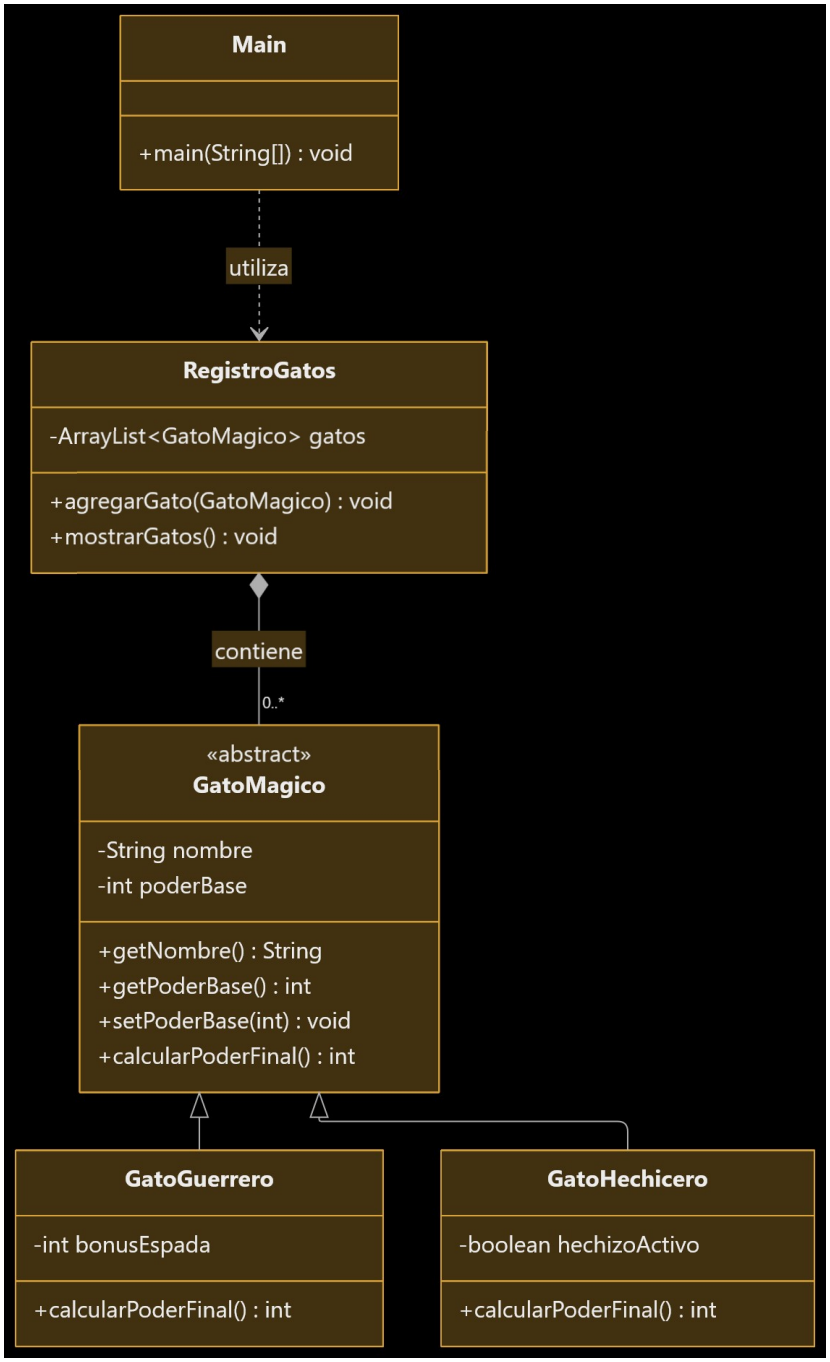
Se aceptan pequeñas diferencias en los textos o en el formato de los números, siempre que se muestren los datos solicitados y los resultados numéricos sean correctos. **La ejecución del programa debe mostrar información similar a:**



– *Diagrama de Clases* –

El sistema deberá organizarse mediante una jerarquía donde `GatoGuerrero` y `GatoHechicero` heredan de la clase abstracta `GatoMagico`.

La clase `RegistroGatos` administra una colección de objetos `GatoMagico`, mientras que `Main` utiliza dicha clase para registrar y mostrar los personajes.



Rúbrica de Evaluación – Kat Kingdom

Tiempo: 60 minutos | Bonificación máxima: 1,00 punto

Criterio	Logrado (100%)	No logrado (0%)	Puntaje máximo	Puntaje obtenido
1. Clase abstracta GatoMagico	Declara correctamente la clase abstracta, atributos privados, constructor, getters y método abstracto.	No implementa la clase o su estructura es incorrecta.	**0,20**	
2. Herencia y constructores	Ambas clases hijas heredan de GatoMagico, declaran sus atributos y utilizan super().	No establece correctamente la herencia.	**0,20**	
3. Sobreescritura de métodos	Ambas clases sobrescriben calcularPoderFinal() y realizan correctamente sus cálculos.	No implementa la sobreescritura o los cálculos son incorrectos.	**0,20**	
4. Validación mediante setter	setPoderBase() valida el valor, lanza IllegalArgumentException y es utilizado por el constructor.	No implementa la validación solicitada.	**0,15**	
5. RegistroGatos y ArrayList	Declara e inicializa ArrayList<GatoMagico> en RegistroGatos y agrega objetos mediante agregarGato().	No implementa la colección ni el método de registro.	**0,10**	
6. Polimorfismo y visualización	Main registra los tres gatos y mostrarGatos() muestra nombre, poder base y poder final mediante polimorfismo.	No demuestra el funcionamiento del registro ni la visualización.	**0,15**	

¡Ánimo Chicos!

¡Cualquier consulta, reclamo, queja, duda que tengan no duden en contactarme!

¡Mándame tu ejercicio y yo te lo reviso!

Correo Institucional**:** **mi.gonzalezt@duocuc.cl**